UK Road Safety Analysis

Processing CSV Data

The initial lines of the code are imports from external modules which will be utilized throughout the Jupyter Notebook. Initially, it uses `Pandas` to parse the CSV file assigns it to a variable (raw_accident_data). It then goes through a series of methods provided by Pandas to process and make changes to the dataset to enhance the analytics outcomes.

In [1]:
```
import matplotlib.pyplot as plt # static scatterplot, heatmap, bar, line
import seaborn as sns # boxplot
import plotly.express as px # interactive chart
import numpy as np # useful for converting to arrays
import tkinter as tk # GUI
from tkinter import ttk # widgets
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg # matplotlib
import pandas as pd # data handling, processing
import sys # checking for correct environment (set to Anaconda)

print("environment:", sys.executable)

raw_accident_data = pd.read_csv('/Users/minsukim/github/finalproject/kaggled
# reading the file I have imported from Kaggle, and then assigning to a vari
raw_accident_data
# printing the raw_accident_data and displaying the contents inside
raw_accident_data.describe(include = 'all')

na_dropped = raw_accident_data.dropna()
# display contents elminating NaN
na_dropped
```

```
environment: /opt/anaconda3/bin/python
```

Out[1]:

	Accident_Index	Accident Date	Day_of_Week	Junction_Control	Junction_Detail	A
195	BS0000196	13-05-2021	Wednesday	Give way or uncontrolled	T or staggered junction	
414	BS0000415	04-09-2021	Friday	Auto traffic signal	Crossroads	
476	BS0000477	06-10-2021	Tuesday	Auto traffic signal	T or staggered junction	
854	BS0000855	24-09-2021	Thursday	Data missing or out of range	Not at junction or within 20 metres	
889	BS0000890	05-10-2021	Monday	Give way or uncontrolled	T or staggered junction	
...	...	...	...	...	...	
307902	BS0307903	26-05-2022	Wednesday	Data missing or out of range	Not at junction or within 20 metres	
307911	BS0307912	23-08-2022	Monday	Data missing or out of range	Not at junction or within 20 metres	
307918	BS0307919	14-11-2022	Sunday	Data missing or out of range	Not at junction or within 20 metres	
307960	BS0307961	21-01-2022	Thursday	Data missing or out of range	Not at junction or within 20 metres	
307972	BS0307973	28-02-2022	Sunday	Give way or uncontrolled	T or staggered junction	

5424 rows × 21 columns

In [2]:

```
# data cleaning for the NaN data
clean_data = na_dropped.reset_index(drop=True)
# resetting so the index are starting from 0
accidentsCleanedData = clean_data
clean_data
# displaying all the columns
print(clean_data.columns)
```

```
Index(['Accident_Index', 'Accident Date', 'Day_of_Week', 'Junction_Control',
       'Junction_Detail', 'Accident_Severity', 'Latitude', 'Light_Condition
s',
       'Local_Authority_(District)', 'Carriageway_Hazards', 'Longitude',
       'Number_of_Casualties', 'Number_of_Vehicles', 'Police_Force',
       'Road_Surface_Conditions', 'Road_Type', 'Speed_limit', 'Time',
       'Urban_or_Rural_Area', 'Weather_Conditions', 'Vehicle_Type'],
      dtype='object')
```

Initial Control Structures and Functions

This block of code explores the dataset by looping through some of the noteworthy datapoints such as severity of the accidents, and road surface conditions. Also included are functions that calculates the average by using python methods, as well as Pandas functions to update the contents in the cells (Time).

In [3]:
```python
# getting the count of severity levels of accidents
serious_count = 0
fatal_count = 0
slight_count = 0

# looping through updated data column Accident_Severity, count by increment
for severity in clean_data["Accident_Severity"]:
    if severity == "Serious":
        serious_count += 1
    if severity == "Fatal":
        fatal_count += 1
    if severity == "Slight":
        slight_count += 1

print("Serious Accident: ",serious_count)
print("Fatal Accident: ",fatal_count)
print("Slight Accident: ",slight_count)

# exploring conditions of the road and weather using the cleaned
dry_count = 0
wet_count = 0

# counting by increments on column "Road_Surface_Conditions"
for road_conditions in clean_data["Road_Surface_Conditions"]:
    if road_conditions == "Dry":
        dry_count += 1
    if road_conditions == "Wet or damp":
        wet_count += 1

print("Dry Conditions: ", dry_count)
print("Wet Conditions: ", wet_count)

unique_weather = clean_data["Weather_Conditions"].unique()
print("this is all the unique types of weather: ", unique_weather)

fine_count = 0
rain_count = 0
```

```
snow_count = 0

for weather_conditions in clean_data["Weather_Conditions"]:
    if weather_conditions == "Fine no high winds":
        fine_count += 1

print(fine_count)

# creating a function to calculate daily average accident count
def average_daily_accidents(clean_data):
    return clean_data.groupby("Accident Date").size().mean()

avg_accidents = average_daily_accidents(clean_data)
print("accidents per day: ", avg_accidents)

# creating a function to change the time format to HH:MM AM/PM format
# will use this for seeing trends of accidents during certain times
def TimeToHours():
    clean_data['Time'] = pd.to_datetime(clean_data['Time'], errors='coerce')
    # overwriting the time cells to 12 hour format
    clean_data['Time'] = clean_data['Time'].dt.strftime('%-I %p')

# function callback to make the modification
TimeToHours()
# return a specified number of rows from the top
clean_data.head()
```

```
Serious Accident:  781
Fatal Accident:  74
Slight Accident:  4569
Dry Conditions:  3332
Wet Conditions:  1694
this is all the unique types of weather:  ['Fine no high winds' 'Raining no
high winds' 'Other'
 'Raining + high winds' 'Snowing no high winds' 'Fine + high winds'
 'Fog or mist' 'Snowing + high winds']
4073
accidents per day:  7.450549450549451
```

```
/var/folders/kx/x55k7dkj7qg_16v_x1prl4qm0000gn/T/ipykernel_15201/3020763214.
py:56: UserWarning: Could not infer format, so each element will be parsed i
ndividually, falling back to `dateutil`. To ensure parsing is consistent and
as-expected, please specify a format.
  clean_data['Time'] = pd.to_datetime(clean_data['Time'], errors='coerce')
```

Out[3]:

	Accident_Index	Accident Date	Day_of_Week	Junction_Control	Junction_Detail	Accide
0	BS0000196	13-05-2021	Wednesday	Give way or uncontrolled	T or staggered junction	
1	BS0000415	04-09-2021	Friday	Auto traffic signal	Crossroads	
2	BS0000477	06-10-2021	Tuesday	Auto traffic signal	T or staggered junction	
3	BS0000855	24-09-2021	Thursday	Data missing or out of range	Not at junction or within 20 metres	
4	BS0000890	05-10-2021	Monday	Give way or uncontrolled	T or staggered junction	

5 rows × 21 columns

Data Visualization

1. Box Plot

The Box Plot uses an external module `Seaborn` to analyze the spread and distribution of accident counts per hour. It creates a boxplot for each road type that displays the distribution of accidents by the hour.

2. Scatter Plot

The Scatter plot is built on `matplotlib` which plots the accidents across the hours of the day with points for each severity. It showcases how accident frequency changes over time of day (mild accidents spiking during morning and evening rush hours)

3. Heatmap

The heatmap creates a matrix counting the accidents by severity and brightness when driving. `Numpy` arrays are integrated to the matplotlib as it calculates Boolean arrays efficiently and creates 2D arrays. It shows a correlation between severity of accidents and light conditions, with factors including car lights, natural light, and darkness.

4. Line chart

The linechart counts the accidents for the top 6 unique weather conditions, mainly due to wide range of conditions that will overlap the visualization. This helps analyze the

trends of accidents across different weather conditions.

5. Bar chart

The bar chart counts the accidents by each leve of severity. It analyzes the overall distribution of accident severity and quickly summarizes whether most accidents are slight, serious, or fatal.

6. Intergrative Scatter Plot

This scatter plot uses the `Plotly` module which allows users to hover, filter or zoom to discover more detailed information of the accident. This plot analyzes the relationship between road types and junctions regarding the accidents. It highlights which junctions are most dangerous in the UK, which is the single carriageway.

Graphical User Interface

The last portion of the code block shows the Tkinter module used to create a GUI of the six data visualization above it. The functions are essentially called once the user clicks the button, which prompts it to display what is embedded in the window. The plots are able be embedded via `FigureCanvasTkAgg` which helps dynamically update the display area and preventing any duplication of the plot.

In [4]:
```
### seaborn boxplot
def boxplot():
    # initial matplotlib states created state bugs so i created subplots so
    fig, ax = plt.subplots(figsize=(10, 5))
    # Group by Road_Type and Time to count accidents per hour
    accidents_per_hour_road = clean_data.groupby(['Road_Type', 'Time']).size
    # print("this is the new table:\n", accidents_per_hour_road.head())

    sns.boxplot(
        # compare distribution of value across road type and accidents
        x='Road_Type',
        y='Accident_Count',
        data=accidents_per_hour_road,
        # palette for differentiating colors
        palette='Set2'
    )

    # axes
    ax.set_title('Distribution of Accidents per Hour by Road Type')
    ax.set_xlabel('Road Type')
    ax.set_ylabel('Number of Accidents per Hour')
    plt.setp(ax.get_xticklabels(), rotation=45)

    display_plot(fig)

### matplotlib scatterplot
```

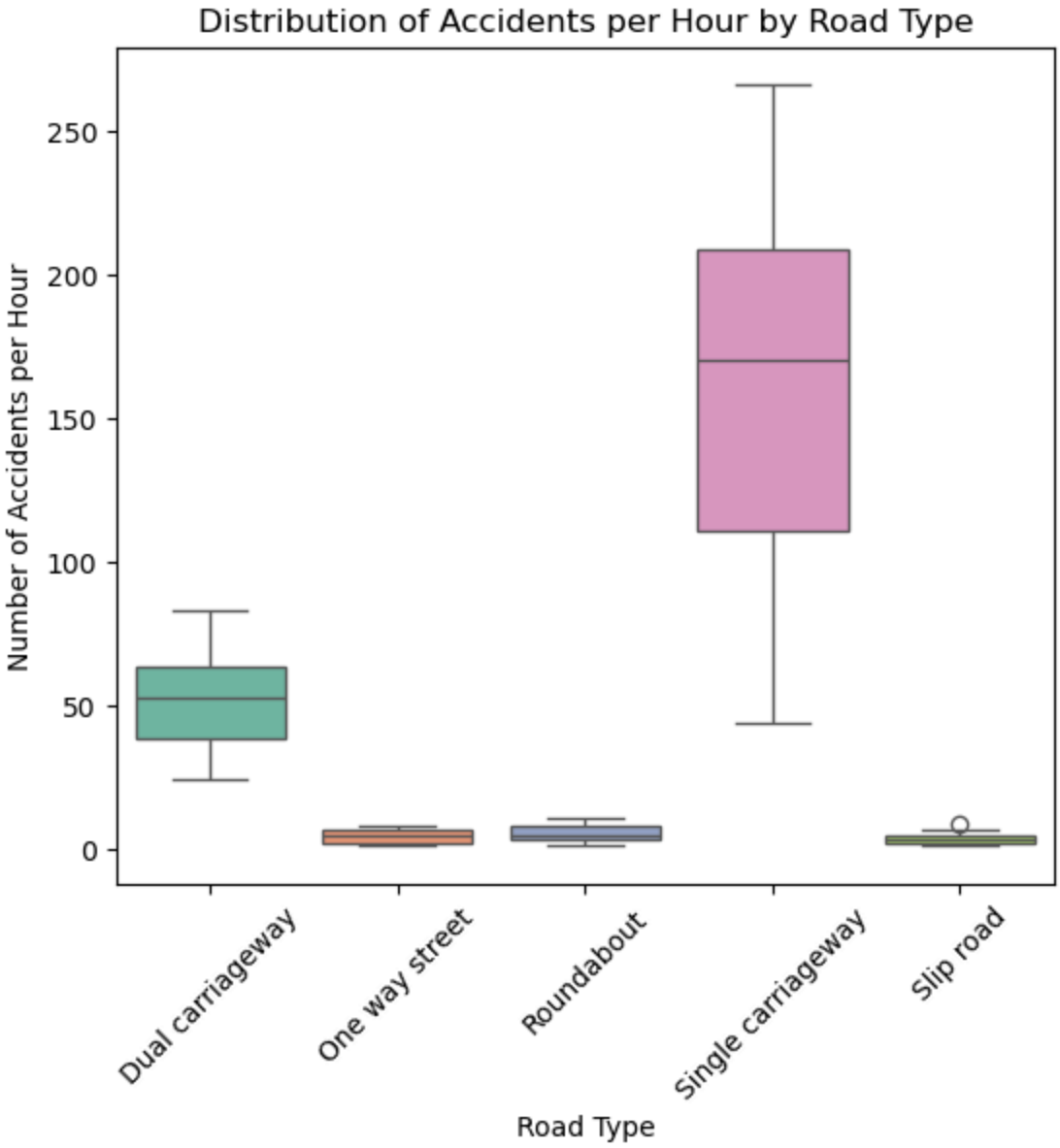
```
def scatterplot():
    # created an array rearrange the time on x axis(strings)
    time_order = [
    "12 AM", "1 AM", "2 AM", "3 AM", "4 AM", "5 AM",
    "6 AM", "7 AM", "8 AM", "9 AM", "10 AM", "11 AM",
    "12 PM", "1 PM", "2 PM", "3 PM", "4 PM", "5 PM",
    "6 PM", "7 PM", "8 PM", "9 PM", "10 PM", "11 PM"
    ]
    # create chart with matplotlib
    fig, ax = plt.subplots(figsize=(10, 5))
    #print(clean_data['Time'].dtype)

    # assigning the value to calculate the accidents by every hour
    # group by hour of the accident
    # unstack to create one column per severity
    accidents_per_hour_severity = (
        clean_data.groupby(
            ['Time', 'Accident_Severity'])
            .size()
            .unstack(fill_value=0)
            # using the time_order I'm re-index to enforce chronological orc
            .reindex(time_order, fill_value=0)
    )
    #print(accidents_per_hour_severity)

    # loop through each severity type
    for severity in accidents_per_hour_severity.columns:
        ax.scatter(
            accidents_per_hour_severity.index,
            accidents_per_hour_severity[severity],
            label=severity
        )

    # Add gridlines
    ax.set_xlabel('Hour of Day')
    ax.set_ylabel('Number of Accidents')
    ax.set_title('Number of Accidents by Hour of Day')
    ax.set_xticks(range(24))
    ax.set_xticklabels(time_order, rotation=90)
    ax.grid(True, linestyle='--', alpha=0.7)
    ax.legend(title='Accident Severity')

    display_plot(fig)

### heatmap matplotlib
def heatmap():
    # after storing heatmap, set up plot
    fig, ax = plt.subplots(figsize=(10, 5))

    # converting csv columns to numpy arrays
    severity = clean_data["Accident_Severity"].to_numpy()
    light = clean_data["Light_Conditions"].to_numpy()

    # getting unique values for rows and columns
    unique_severity = np.unique(severity)
    unique_light = np.unique(light)
```

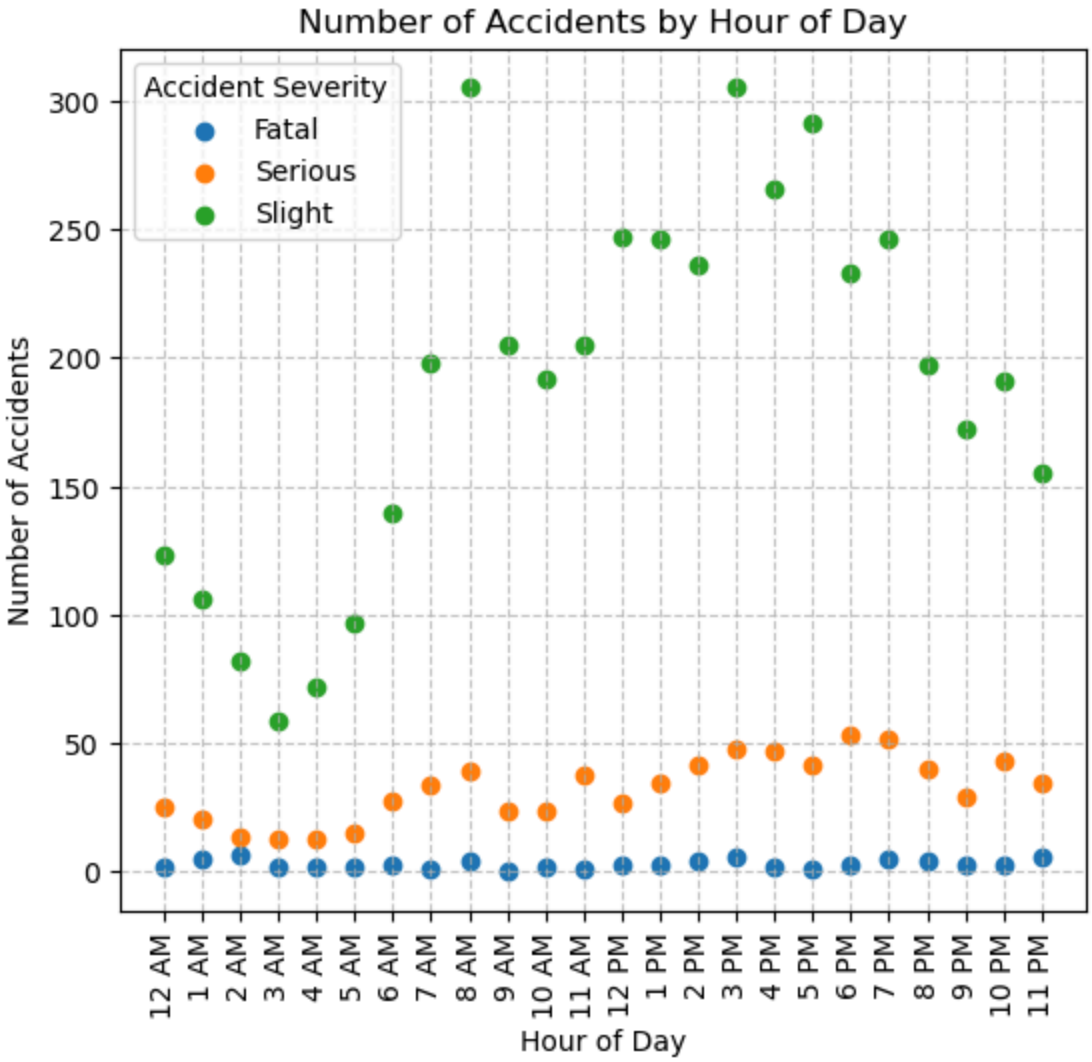
```
    #print(unique_light, unique_severity)

    # setting up an empty heatmap
    heatmap_data = np.zeros((len(unique_severity), len(unique_light)))

    # filling in the heatmap
    # loopings through the array of unique severities
    for i, sev in enumerate(unique_severity):
        # inner looping through the unique light conditions
        # checking every severity through each light condition
        for j, l in enumerate(unique_light):
            # Count accidents for each severity and light condition
            #np.sum counts the array (true or false) for severity and light
            heatmap_data[i, j] = np.sum((severity == sev) & (light == l))

    # displays 2D data as image, cell color to red,
    cax = ax.imshow(heatmap_data, cmap="Reds")
    # add colorbar for the heat scale bar
    fig.colorbar(cax, ax=ax, label="Number of Accidents")
    # positioning each x axis label at the correct index and aligning text t

    # setting the tick labels
    ax.set_xticks(np.arange(len(unique_light)))
    ax.set_xticklabels(unique_light, rotation=45, ha="right")
    ax.set_yticks(np.arange(len(unique_severity)))
    ax.set_yticklabels(unique_severity)

    #setting x and y labels
    ax.set_xlabel("Light Conditions")
    ax.set_ylabel("Accident Severity")
    ax.set_title("Severity vs Light Conditions")

    display_plot(fig)

### line chart
def linechart():
    fig, ax = plt.subplots(figsize=(10, 5))

    # getting the top 6 from the unique values of weather
    weather_counts = clean_data["Weather_Conditions"].value_counts().head(6)
    # print("Top Weather Conditions:")
    # print(weather_counts)

    # setting line chart for weather conditions when accidents happened
    ax.plot(
            # x positions
            range(len(weather_counts)),
            weather_counts.values,
            marker='o',
            linestyle='-',
            linewidth=3
        )

    ax.set_title("Weather Conditions During Accidents")
    # setting the label rotation and alignments
    ax.set_xlabel("Weather Condition")
```

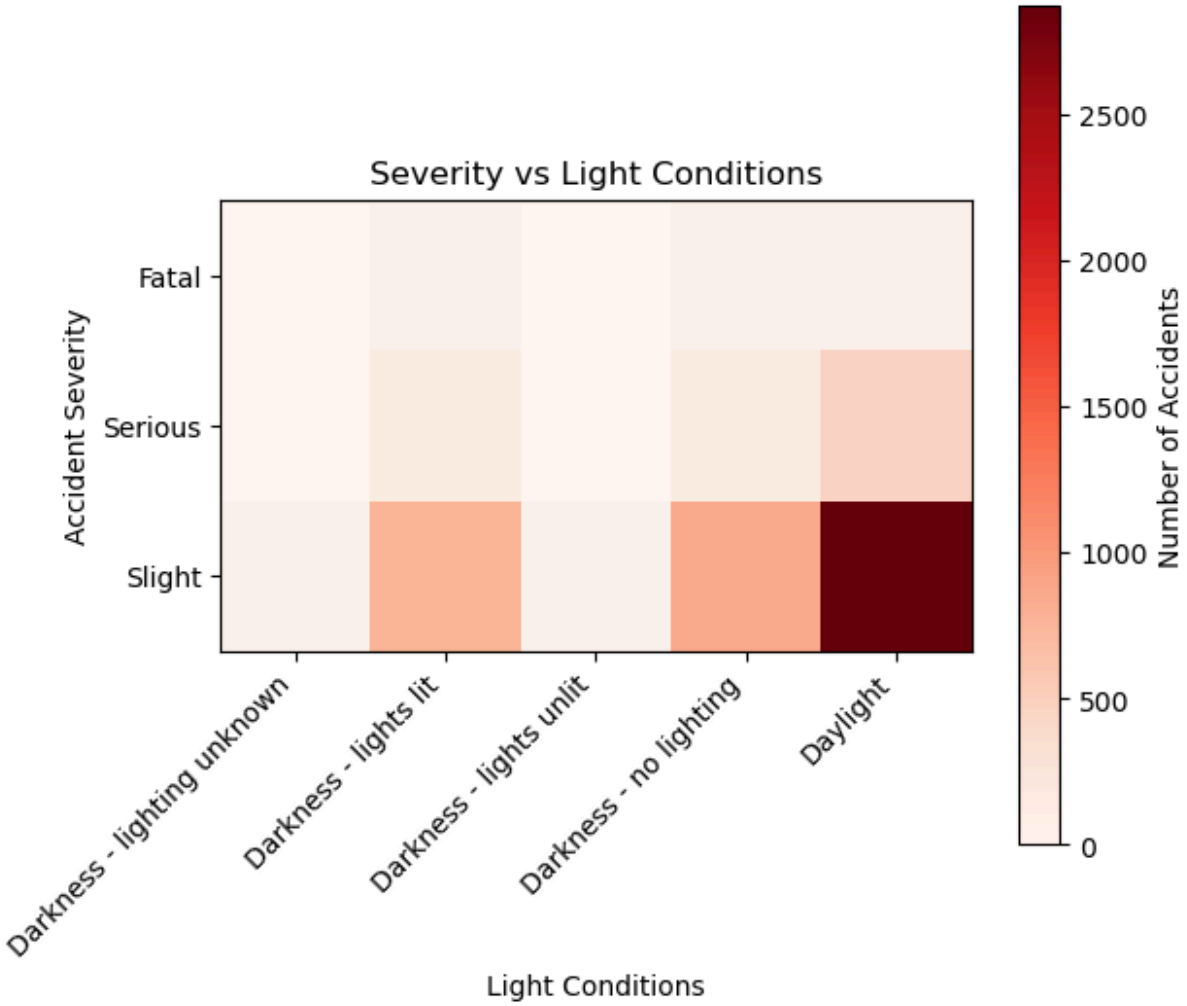
```
    ax.set_ylabel("Accidents")
    ax.set_xticks(range(len(weather_counts)))
    ax.set_xticklabels(weather_counts.index, rotation=45, ha='right')
    fig.tight_layout()

    display_plot(fig)
    # plt.savefig("weatherConditionsDuringAccidents.png")

### bar chart
def barchart():
    fig, ax = plt.subplots(figsize=(10, 5))

    # using pandas value counts to return counts of unique values
    severity_counts = clean_data["Accident_Severity"].value_counts()
    # print("count of severity:", severity_counts)

    # setting up simple bar chart with severity level as x axis and number c
    ax.bar(severity_counts.index, severity_counts.values)

    ax.set_title("Accident Counts by Severity")
    ax.set_xlabel("Severity")
    ax.set_ylabel("Number of Accidents")
    fig.tight_layout()
    display_plot(fig)
    #plt.savefig("accidentCountsbySeverity.png")
    #plt.show()

def interactive_scatter():
    ploty_fig = px.scatter(
        # grouping by Road_Type and Junction Detail
        # size counts the # of accidents,
        # turn the grouped data back into clean_data and names the count col
        clean_data.groupby(['Road_Type', 'Junction_Detail']).size().reset_ir
        x='Road_Type',
        y='Accident_Count',
        # color by junction detail
        color='Junction_Detail',
        # size visualization by accident count
        size='Accident_Count',
        # interactive by hoveringing
        hover_data=['Road_Type', 'Junction_Detail', 'Accident_Count'],
        title="Accidents by Road Type and Junction Detail"
    )

    # built in filters to enable dynamic updates based on dropdown selectior
    ploty_fig.update_layout(
        title_font_size=18,
        hovermode="closest",
        showlegend=True,
        legend_title_text="Junction_Detail"
    )
    #plotly.offline.plot(ploty_fig, filename='scatterPlot.html')
    ploty_fig.show()

# created the function to eliminate any duplicates and dynamically update th
def display_plot(fig):
```

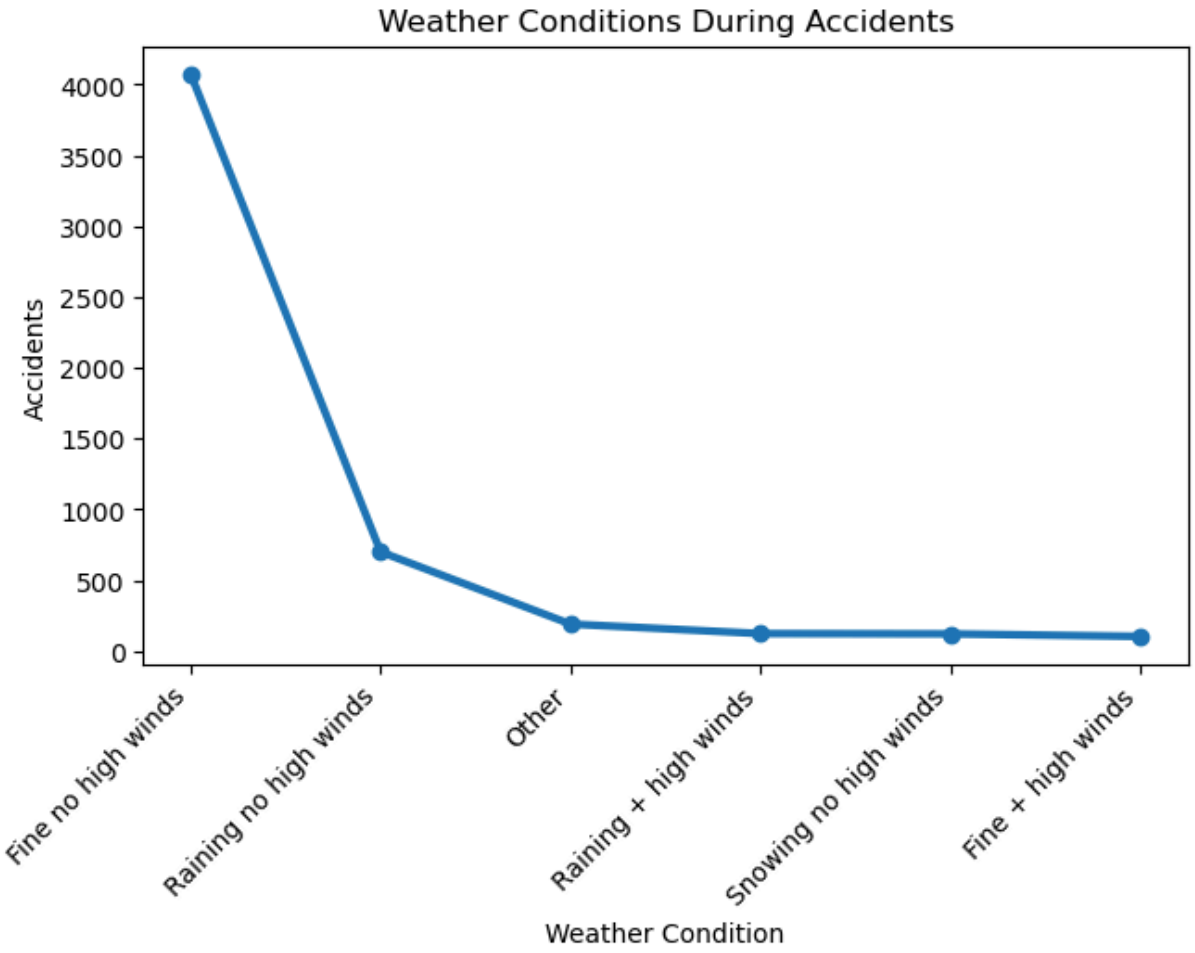
```
    for widget in frame_plot.winfo_children():
        widget.destroy()  # Clear previous plot
    canvas = FigureCanvasTkAgg(fig, master=frame_plot)
    canvas.draw()
    canvas.get_tk_widget().pack(fill="both", expand=True)

root = tk.Tk()
root.title("UK Accident Dashboard")
root.geometry("1000x600")

# Frame for buttons (left side)
frame_buttons = ttk.Frame(root)
frame_buttons.pack(side="left", padx=20, pady=20)

# Frame for plot display (right side)
frame_plot = ttk.Frame(root)
frame_plot.pack(side="right", padx=20, pady=20, fill="both", expand=True)

# buttons for different plots in the bottom of the frame
tk.Button(frame_buttons, text="Boxplot", command=boxplot).pack(pady=5, fill=
tk.Button(frame_buttons, text="Scatterplot", command=scatterplot).pack(pady=
tk.Button(frame_buttons, text="Heatmap", command=heatmap).pack(pady=5, fill=
tk.Button(frame_buttons, text="Line Chart", command=linechart).pack(pady=5,
tk.Button(frame_buttons, text="Bar Chart", command=barchart).pack(pady=5, fi
tk.Button(frame_buttons,text="Interactive Plotly Chart (Jupyter Notebook)",c

# running the application
root.mainloop()
```

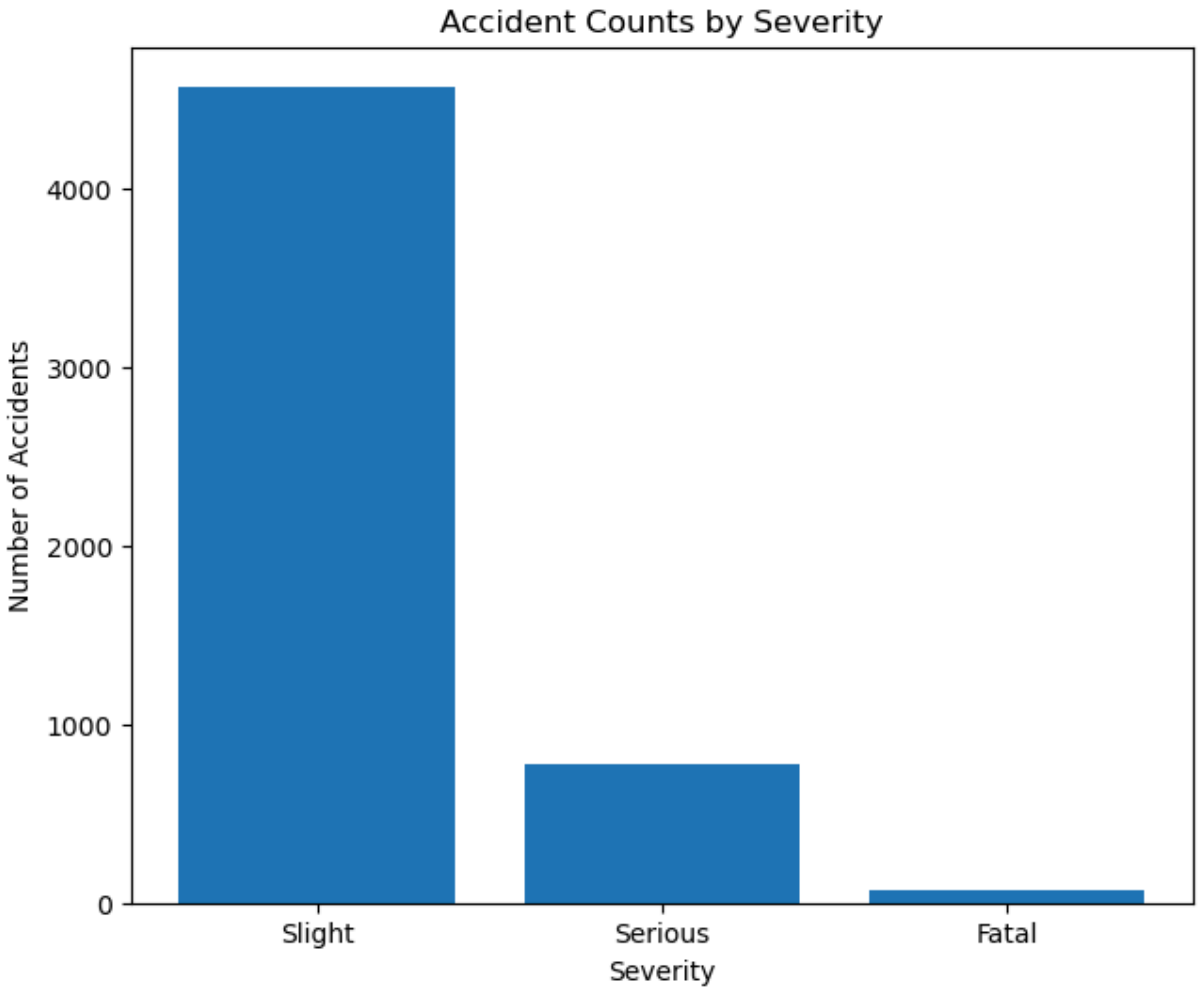
```
/var/folders/kx/x55k7dkj7qg_16v_x1prl4qm0000gn/T/ipykernel_15201/2669284938.
py:9: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed
in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the
same effect.

  sns.boxplot(
```


Distribution of Accidents per Hour by Road Type
Number of Accidents per Hour
0
50
100
150
200
250
Dual carriageway
One way street
Roundabout
Single carriageway
Slip road
Road Type


Number of Accidents by Hour of Day
Accident Severity
Fatal
Serious
Slight
Number of Accidents
0
50
100
150
200
250
300
12 AM
1 AM
2 AM
3 AM
4 AM
5 AM
6 AM
7 AM
8 AM
9 AM
10 AM
11 AM
12 PM
1 PM
2 PM
3 PM
4 PM
5 PM
6 PM
7 PM
8 PM
9 PM
10 PM
11 PM
Hour of Day


Severity vs Light Conditions
Accident Severity
Fatal
Serious
Slight
Darkness - lighting unknown
Darkness - lights lit
Darkness - lights unlit
Darkness - no lighting
Daylight
Light Conditions
Number of Accidents
2500
2000
1500
1000
500
0


Weather Conditions During Accidents
Accidents
4000
3500
3000
2500
2000
1500
1000
500
0
Fine no high winds
Raining no high winds
Other
Raining + high winds
Snowing no high winds
Fine + high winds
Weather Condition


Accident Counts by Severity
Number of Accidents
0
1000
2000
3000
4000
Slight
Serious
Fatal
Severity